Trabajo Práctico Integrador Grupal

Gladiadores de Consola

Torneo por Turnos

Modalidad: grupal, equipos de 5 integrantes.
Lenguaje y entorno: C# en Visual Studio (aplicación de consola).
Entrega: repositorio de GitHub + README, el día ____ / ____ / _____.
Defensa oral: individual, en clase, el día ____ / ____ / _____.

1. De qué se trata

Dos jugadores se sientan frente a la misma computadora. Cada uno arma su equipo de gladiadores, les reparte puntos de vida, ataque y defensa, y después se enfrentan en la arena por turnos, eligiendo en cada jugada a quién atacar o cuándo conviene recuperarse. El combate termina cuando un equipo se queda sin luchadores en pie, y ahí el programa muestra quién ganó, quién fue el mejor gladiador del torneo y cómo quedó cada uno.

Es un juego completo, de punta a punta, que integra todo lo que vieron en la materia hasta ahora. No hay una única forma correcta de resolverlo: parte de lo que se evalúa es cómo el equipo decide organizar los datos y dividir el programa.

2. Objetivos y temas evaluados

Al terminar el trabajo, cada integrante debería poder:

- Analizar un problema de varias partes y decidir cómo dividirlo antes de escribir código.
- Elegir cómo guardar y relacionar los datos de muchos elementos que comparten características.
- Diseñar funciones con una responsabilidad clara, decidiendo qué reciben y qué devuelven.
- Validar todo lo que ingresa el usuario para que el programa no avance con datos incorrectos.
- Calcular resultados a partir de los datos acumulados durante el programa.
- Trabajar en equipo sobre un mismo código usando Git y GitHub.

Temas que se ponen en juego: tipos de datos y operadores; entrada y salida por consola; estructuras condicionales y repetitivas; contadores y acumuladores; arreglos unidimensionales; funciones; buenas prácticas de escritura de código; control de versiones.

3. Organización del equipo

Los equipos son de **5 integrantes**. La forma de repartirse el trabajo la decide el equipo, pero tiene que cumplir con lo siguiente:

- **Diseño antes de codificar.** Antes de escribir código, el equipo acuerda qué datos va a guardar, en qué partes se divide el programa y quién se encarga de cada una. Ese acuerdo queda escrito en el README.

- **Todos programan.** Cada integrante es responsable de al menos una parte identificable del programa. No alcanza con probar o documentar.
- **Un solo repositorio.** El proyecto vive en un repositorio de GitHub compartido, y cada integrante sube su trabajo con sus propios commits, con mensajes que digan qué se hizo.
- **Pruebas cruzadas.** Cada parte la prueba un integrante distinto del que la programó.
- **Todos entienden todo.** En la defensa oral cualquier integrante puede ser consultado sobre cualquier parte del programa, no solo la propia.

El README del repositorio tiene que incluir:

  - Nombre del equipo e integrantes.
  - Qué parte del programa hizo cada integrante y quién la probó.
  - Cómo se juega (una explicación breve para alguien que nunca lo usó).
  - Las decisiones de diseño que tomaron y por qué (cómo guardaron los datos, cómo dividieron el programa).

4. Requisitos funcionales

Esto es lo que el programa tiene que hacer. Los textos y el diseño de las pantallas son libres, siempre que la información sea clara para quien juega.

4.1 Configuración de la partida

- Al iniciar, el programa pide la modalidad de combate. Solo se aceptan **1 vs 1, 2 vs 2, 3 vs 3 o 5 vs 5**. Cualquier otro valor se rechaza y se vuelve a pedir hasta que sea válido.
- La cantidad de gladiadores de cada equipo depende de la modalidad elegida. El programa tiene que funcionar correctamente en las cuatro modalidades.

4.2 Creación de los equipos

- Primero arma su equipo el Jugador 1 y después el Jugador 2. Cada jugador ingresa un **nombre para su equipo**.
- Para cada gladiador se ingresa **Nombre, Vida, Ataque y Defensa**.
- El nombre no puede quedar vacío ni repetirse con el de ningún otro gladiador de la partida, de ninguno de los dos equipos.
- Cada gladiador tiene **40 puntos** para repartir entre sus tres atributos, respetando estos límites:

Atributo	Mínimo	Máximo	Para qué sirve
Vida	10	25	Cuánto daño aguanta antes de quedar fuera de combate.
Ataque	5	20	Cuánto daño puede hacer al golpear.
Defensa	1	10	Cuánto daño frena de cada golpe que recibe.

- Si un valor está fuera de su rango, se rechaza y se vuelve a pedir.
- La suma **Vida + Ataque + Defensa tiene que dar exactamente 40**. Si da más o menos, el programa avisa cuántos puntos se usaron y pide repartir los tres atributos de nuevo.
- La vida con la que se crea cada gladiador es su **vida máxima** para todo el combate.

4.3 El combate

- El combate es por turnos. **Empieza el Jugador 1** y después se alternan, una acción por turno.
- Al comienzo de cada turno se muestra el número de turno y el estado de los dos equipos: cada gladiador con su vida actual y su vida máxima, indicando cuáles están fuera de combate.
- El jugador activo elige una de dos acciones:

  Acción 1: Atacar

  - Elige uno de sus gladiadores para atacar y un gladiador rival para recibir el golpe. Los dos tienen que estar en pie (vida mayor a 0). Si elige uno fuera de combate o un número que no existe, se vuelve a pedir.
  - **Daño = Ataque del atacante − Defensa del defensor.** Si el resultado es menor que 1, el daño es 1: todo golpe hace al menos un punto.
  - La vida nunca queda negativa. Si el daño es mayor que la vida que le queda al rival, el rival queda en 0.
  - Cada gladiador lleva la cuenta del **daño total que causó** en el torneo. Se suma solamente la vida que realmente le quitó al rival (si le quedaban 4 de vida y el golpe era de 10, se suman 4).
  - Cuando un gladiador llega a 0, el programa informa que quedó fuera de combate.

  Acción 2: Recuperarse

  - Elige uno de sus gladiadores en pie, que recupera **5 puntos de vida** sin superar su vida máxima.
  - Cada gladiador puede recuperarse como máximo **2 veces** en todo el combate.
  - Los gladiadores fuera de combate no pueden recuperarse.
  - Si el gladiador elegido ya tiene la vida completa o ya usó sus dos recuperaciones, el programa avisa y el jugador vuelve a elegir acción. **No pierde el turno.**

- El combate termina en cuanto todos los gladiadores de un equipo quedan fuera de combate. El otro equipo es el ganador.

Ejemplos de cálculo de daño

Atacante	Defensor	Cálculo	Resultado
Ataque 15	Defensa 5, Vida 18	15 − 5 = 10	El defensor queda con 8. Al atacante se le suman 10.
Ataque 8	Defensa 10, Vida 12	8 − 10 = −2 → 1	El defensor queda con 11. Al atacante se le suma 1.
Ataque 20	Defensa 3, Vida 4	20 − 3 = 17	El defensor queda en 0 (fuera de combate). Al atacante se le suman 4.

4.4 Reporte final

Cuando termina el combate, el programa muestra:

- **El equipo ganador** y en cuántos turnos se resolvió el combate.
- **El MVP del torneo:** el gladiador, de cualquiera de los dos equipos, que más daño causó en total, con ese valor. Si hay empate, se muestran todos los empatados.

- **El promedio de vida restante del equipo ganador**, con dos decimales. Entran en el promedio todos los gladiadores del equipo, y los que quedaron fuera de combate cuentan con vida 0. Junto al promedio se indica cuántos gladiadores quedaron en pie sobre el total del equipo.
- **La tabla final** con todos los gladiadores de la partida: equipo, nombre, vida final y vida máxima, ataque, defensa, daño causado y estado (en pie o fuera de combate).

Pantallas de ejemplo

Solo como referencia de qué información tiene que aparecer. No hace falta copiar el formato.

```
================ TURNO 5 ================

[Los Leones]
  1. Máximo   Vida 15/20
  2. Brisa    Vida 0/12   (fuera de combate)

[Hierro Sur]
  1. Tauro    Vida 9/25
  2. Pinta    Vida 14/18

Juega Los Leones (Jugador 1)
Acción: 1) Atacar  2) Recuperarse: 1
Tu gladiador que ataca (número): 2
Brisa está fuera de combate. Elegí otro.
Tu gladiador que ataca (número): 1
Gladiador rival que recibe (número): 1
Máximo golpea a Tauro y le quita 9 de vida.
Tauro queda fuera de combate.
```

```
================ FIN DEL TORNEO ================
Gana Los Leones (Jugador 1) en 14 turnos.
MVP del torneo (41 de daño causado): Máximo
Promedio de vida restante de Los Leones: 3.50 (sobrevivientes: 1 de 2)

EQUIPO       GLADIADOR   VIDA    ATQ  DEF  DAÑO  ESTADO
Los Leones   Máximo      7/20    15   5    41    En pie
Los Leones   Brisa       0/12    18   10   22    Fuera de combate
Hierro Sur   Tauro       0/25    10   5    16    Fuera de combate
Hierro Sur   Pinta       0/18    14   8    27    Fuera de combate
```

5. Restricciones técnicas

El trabajo tiene que resolverse con las herramientas que vimos en la materia. El objetivo es que cada decisión la tome el equipo, no una librería.

Se puede usar	No se puede usar
• Variables y tipos de datos primitivos (`int`, `double`, `string`, `bool`, etc.). • Operadores aritméticos, relacionales y lógicos. • Entrada y salida por consola. • Estructuras condicionales y repetitivas vistas en clase. • Arreglos unidimensionales. • Funciones propias, con o sin parámetros y con o sin retorno. • Constantes para los valores fijos del juego. • Conversiones de texto a número y formato de decimales.	• Clases propias, structs o cualquier recurso de programación orientada a objetos. • Colecciones como `List<>`, `Dictionary<>` o similares. • LINQ (`.Max()`, `.Sum()`, `.Where()`, etc.). • Métodos ya hechos de arreglos: `Array.Sort`, `Array.IndexOf`, `Array.Resize`, `.Contains()` y parecidos. • Arreglos de más de una dimensión. • Variables declaradas fuera de las funciones para compartir datos entre ellas (las constantes sí se permiten). • Números al azar, librerías externas o paquetes NuGet. • `goto`.

- El programa tiene que estar **dividido en funciones**. Un `Main` que concentra toda la lógica no cumple con el trabajo.
- El programa tiene que compilar y ejecutarse sin errores en Visual Studio.
- Si se usa algo de la columna de la derecha, el criterio de evaluación correspondiente se califica como insuficiente.

Desafíos opcionales

No son obligatorios. Suman en la nota si el resto del trabajo está completo y funcionando.

- Que el programa no se cierre con un error si en un lugar donde se espera un número el usuario escribe letras.
- Que al comparar nombres repetidos no importen las mayúsculas ("Tauro" y "tauro" cuentan como el mismo nombre).
- Al terminar, ofrecer jugar una revancha con los mismos gladiadores en plena vida, sin volver a cargarlos.

6. Evaluación

La nota tiene una parte grupal (el programa y el repositorio) y una parte individual (la defensa oral).

Criterio	Peso	Qué se mira
Funcionamiento y reglas del juego	25%	El juego se puede jugar completo en las cuatro modalidades y respeta todas las reglas de la sección 4.
Validaciones	15%	Ningún dato incorrecto pasa: modalidad, nombres, rangos, suma de puntos, elección de gladiadores y de acciones.
División en funciones	20%	Cada función hace una sola cosa, tiene un nombre que lo dice y recibe por parámetro lo que necesita. No hay código repetido que podría ser una función.
Organización de los datos	15%	Los datos de cada gladiador se mantienen relacionados y consistentes durante todo el programa, sin importar la modalidad.
Reporte final	10%	MVP (con empates), promedio y tabla final calculados correctamente.
Buenas prácticas	5%	Nombres claros, indentación prolija, comentarios donde aportan, constantes en lugar de números sueltos.
Trabajo en equipo y repositorio	10%	Commits de los cinco integrantes, mensajes claros, README completo.

Defensa oral. Cada integrante explica partes del código elegidas por el docente, propias o ajenas, y puede tener que hacer una modificación pequeña en el momento. La nota individual parte de la nota grupal y se ajusta según la defensa: quien no pueda explicar el código del trabajo no aprueba aunque el programa funcione.